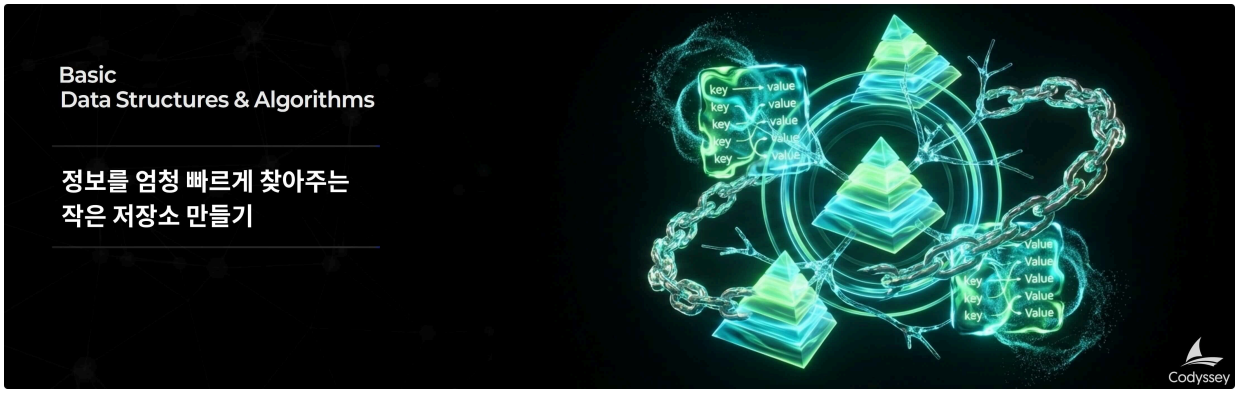
분야 AI/SW 기초 구분 자료구조와 알고리즘 학습시간 80시간 필수/선택 필수

정보를 엄청 빠르게 찾아주는 작은 저장소 만들기

문제기술

기술적 설명



1. 미션 소개

Redis를 써봤는데 왜 이렇게 빠른지 설명해보라고 하면 막히는 분들이 많습니다. 이유가 있습니다. 내부의 자료구조를 직접 구현해본 사람이 드물기 때문입니다. 해시맵, 이중 연결 리스트, 힙을 밑바닥부터 짜면서 LRU와 TTL이 어떻게 동작하는지 손으로 확인합니다.

Redis는 전 세계에서 가장 널리 사용되는 In-Memory Key-Value 데이터 저장소입니다. 캐시, 메시지 브로커, 세션 저장소 등 다양한 용도로 활용되며, 그 핵심에는 효율적인 자료구조가 있습니다.

이번 미션에서는 Redis의 핵심 기능을 직접 구현하며 CLI 기반 Mini Redis를 완성합니다. 해시맵, 이중 연결 리스트, 힙 같은 자료구조를 밑바닥부터 구현하면서 평소 당연하게 사용하던 내장 자료형의 내부 동작 원리를 깊이 이해하게 됩니다.

또한, 실제 Redis가 메모리 제한 환경에서 어떻게 LRU 방식으로 오래된 데이터를 자동 제거하고, TTL을 통해 만료 시간을 관리하는지 직접 구현하며 체득합니다. 이 경험은 이후 알고리즘 학습, 코딩 테스트, 실제 서비스 성능 최적화로 확장될 수 있습니다.

2. 최종 결과물

다음 기능이 정상 동작하는 CLI 기반 Mini Redis 프로그램 1개를 완성한다.

1. String 타입 기본 명령어 (6개)
    - SET : 키에 값 저장 (성공 시 내부적으로 LRU 추적 업데이트)
    - GET : 키의 값 조회 (성공 시 내부적으로 LRU 추적 업데이트)

- DEL : 키 삭제
- EXISTS : 키 존재 여부 확인
- DBSIZE : 전체 키 개수 반환
- KEYS : 전체 키 목록 출력 (패턴 매칭은 구현하지 않음)

2. 메모리 관리 명령어 (2개)

- CONFIG SET maxmemory : 최대 메모리 제한 설정 (바이트 단위)
- INFO memory : 현재 메모리 사용량, 제한, 제거된 키 개수 확인

3. TTL 관리 명령어 (2개)

- EXPIRE : 키의 만료 시간 설정 (초 단위)
- TTL : 키의 남은 만료 시간 조회

4. CLI 인터페이스

- 사용자가 명령어를 입력하면 즉시 실행 결과를 확인할 수 있는 REPL 환경
- 명령어 파싱, 실행, 결과 출력이 반복적으로 동작

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- 해시맵의 해시 함수와 충돌 해결 방식(체이닝)을 구현 코드를 기반으로 설명할 수 있다.
- 이중 연결 리스트와 해시맵을 조합하여 O(1) LRU 추적이 가능한 이유를 설명할 수 있다.
- 힙이 TTL 만료 시간 관리에 적합한 이유를 설명할 수 있다.
- 메모리 제한 환경에서 LRU 정책으로 데이터를 제거하는 전체 흐름(used_memory 갱신 포함)을 설명할 수 있다.

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. 기본 자료구조 직접 구현 (내장 Key-Value 컬렉션으로 대체 금지)

- 이중 연결 리스트
  - 노드 구조: prev , next , data 필드

    - 주요 메서드: `insert_front`, `insert_back`, `remove_front`, `remove_back`, `remove_node`, `move_to_front`
    - 모든 삽입/삭제/이동 연산은 O(1)이어야 한다

- 해시맵 (체이닝 방식)
    - 주요 메서드: `put`, `get`, `remove`, `contains`, `keys`, `size`
    - 해시 함수는 직접 설계해야 한다
    - 충돌 해결은 체이닝 방식으로 구현해야 한다 (권장: 이중 연결 리스트 재사용)
    - 로드 팩터 0.75 초과 시 버킷을 2배 확장해야 한다

- 힙 (최소 힙)
    - 주요 메서드: `push`, `pop`, `peek`, `size`
    - `_heapify_up`, `_heapify_down` 구현 필요
    - TTL 만료 관리를 위해 `(expire_at, key)` 형태의 요소를 다룰 수 있어야 한다

2. String 타입 명령어 (Redis 스타일 출력)

- 공통 규칙
    - 키 기반 명령어는 실행 전 “만료 여부”를 먼저 확인할 수 있어야 한다 (만료된 키는 삭제 후 ‘없는 키’처럼 처리).
    - 출력은 Redis 스타일을 따른다: `OK`, `(nil)`, `(integer) N`, `(error) ...`

- SET key value
    - 성공 시 `OK`
    - 메모리 초과 시 LRU 제거를 수행한다(세부 규칙은 3번 참조)
    - 기존 키를 덮어쓰는 경우: 기존 TTL은 “초기화(삭제)”한다

- GET key
    - 키가 없거나 만료된 경우 `(nil)`
    - 존재하는 경우 `"value"` 형태로 반환
    - 반환이 성공한 경우에만 LRU를 갱신한다 (만료로 삭제된 경우는 갱신하지 않음)

- DEL key
    - 삭제 성공 시 `(integer) 1`, 없으면 `(integer) 0`

- 삭제 시 LRU/TTL 관련 구조에서도 해당 엔트리를 함께 제거해야 한다

- EXISTS key
  - 존재하면 (integer) 1 , 없으면 (integer) 0
- DBSIZE
  - 현재 저장된 키 개수를 (integer) N 으로 반환
- KEYS
  - 전체 키 목록을 배열 형태로 출력 (정렬/순서는 요구하지 않음)
  - 예시:
    a. "user:2"
    b. "user:3"
  - 키가 없으면 (empty array) 같은 형태로 비어 있음을 표현해도 된다

3. 메모리 관리 + LRU 자동 제거

- CONFIG SET maxmemory bytes
  - bytes 는 0 이상의 정수
  - 0은 “무제한”으로 간주한다
  - 성공 시 OK , 정수 파싱 실패 시 에러 표준을 따른다
- INFO memory
  - 아래 3개 항목을 최소 포함해 출력한다(표현 형태는 동일하면 됨): used_memory:<number> maxmemory:<number> evicted_keys:<number>
- used_memory 산정 기준(공식)
  - used_memory = Σ(len(utf8(key)) + len(utf8(value)))
  - 자료구조(노드/포인터/버킷 등) 오버헤드는 계산에서 제외한다
- LRU 제거 규칙
  - maxmemory > 0 이고, SET 이후 used_memory가 maxmemory를 초과하면 used_memory가 maxmemory 이하가 될 때까지 “가장 오래 사용되지 않은 키(LRU)”부터 제거한다

    - 제거된 키는 `evicted_keys` 에 누적 카운트한다
    - 만약 “단일 엔트리(키+값)” 자체가 maxmemory를 초과한다면:
        - 저장하지 않고 에러(OOM)를 출력한다

4. TTL 관리 (힙 기반)

- `EXPIRE key seconds`
    - key가 없으면 `(integer) 0`
    - seconds가 0 이하라면 “즉시 만료”로 처리해도 된다(존재하면 삭제 후 `(integer) 1`)
    - 정상 설정 시 `(integer) 1`
- `TTL key`
    - key가 없으면 `(integer) -2`
    - key는 존재하지만 만료 시간이 없으면 `(integer) -1`
    - 만료 시간이 있으면 남은 초를 `(integer) N` 으로 반환
- TTL/LRU 엣지 케이스 최소 규칙(명시)
    - 만료된 키는 GET 시 먼저 삭제 후 `(nil)` 을 반환하며, LRU 갱신은 하지 않는다
    - SET이 기존 키를 덮어쓸 때 TTL은 초기화(삭제)한다
    - EXPIRE를 없는 키에 호출하면 `(integer) 0` 이다
    - DEL은 데이터/TTL/LRU 모든 구조에서 엔트리를 함께 제거한다
- 구현 방식은 “힙을 통해 가장 빠른 만료를 빠르게 찾을 수 있어야 한다”는 목표를 만족하면 된다 (예: lazy deletion 전략 등은 구현 선택)

5. 에러 처리 표준 + CLI 인터페이스

- CLI
    - `mini-redis>` 프롬프트 출력
    - 사용자 입력을 읽고 명령어 파싱 및 실행
    - `exit` 또는 `quit` 으로 종료 가능
- 에러 출력(표준 형식 예시)
    - 잘못된 명령: (error) ERR unknown command '<cmd>'

    - 인자 개수 오류: (error) ERR wrong number of arguments for '<cmd>' command
    - 정수 파싱 실패: (error) ERR value is not an integer or out of range
    - 메모리 초과(OOM): (error) OOM command not allowed when used_memory > 'maxmemory'

- 값 파싱
    - 예시처럼 "Alice" 같은 따옴표 입력을 허용한다(구현 난이도에 따라 단순 규칙으로 처리 가능)
    - 최소 요구: 공백이 없는 값 / 큰따옴표로 감싼 값 둘 중 하나 방식은 지원

5. 보너스 과제 (선택)

1. 동적 배열 직접 구현
    - 동적 배열을 직접 구현하고 append/get/set/remove 및 capacity 2배 확장 로직을 포함한다
    - 해시맵 버킷 테이블 확장/힙 내부 저장소에 “배열 확장” 개념을 적용할 수 있다
2. 스택/큐/덱 이해와 활용
    - 스택/큐/덱의 개념을 조사하고 STACK_QUEUE_DEQUE.md로 문서화한다
    - Pub/Sub(보너스 5)나 커맨드 히스토리 같은 큐 기반 기능의 기반이 된다
3. 이진 트리와 순회 알고리즘
    - 이진 트리 구현 및 전위/중위/후위/레벨 순회를 구현한다
    - “힙이 완전 이진 트리를 배열로 표현한다”는 관점을 더 단단히 만든다
4. 이진 탐색 트리(BST)
    - BST 삽입/탐색/삭제 및 중위 순회 정렬 결과를 구현한다
    - 추후 “키 정렬/범위 조회” 같은 확장 기능의 기반이 된다
5. Pub/Sub 기능 구현
    - PUBLISH , SUBSCRIBE 명령어를 추가하고 채널 기반 메시징을 구현한다
    - 구현한 연결 리스트를 메시지 큐(구독자별 버퍼 등)로 재활용할 수 있다

개발환경

6. 개발 환경

- Python 3.8 이상

제약조건

7. 제약 사항

- 라이브러리/내장 자료형 제한(학습 목적)
    - dict , set , collections 사용 금지
    - 단, “고정 길이 배열/인덱스 접근” 수준의 저장소가 필요하다면 제한적으로 사용할 수 있는 구현 방식을 선택하되, 내장 컬렉션으로 해시맵/캐시를 대체하는 방식은 금지한다 (예: dict로 put/get 구현 금지).
- 구조
    - 각 자료구조(해시맵/이중 연결 리스트/힙)는 독립된 모듈/파일로 분리한다
    - 핵심 클래스/함수에는 주석 또는 docstring을 작성한다
- 기능 범위
    - 네트워크 통신은 구현하지 않는다(오직 CLI)
    - 데이터 영속성(파일 저장)은 구현하지 않는다
    - Redis의 복잡 자료형(List/Set/Sorted Set)은 구현하지 않는다
    - 멀티스레딩/락 같은 동시성 처리는 요구하지 않는다

Test Case

8. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 실행 예시는 달라도 되지만, 평가를 위해 최대한 이해하기 쉬운 형태로 개발한다.

- 실행 예시

```CSS
mini-redis> CONFIG SET maxmemory 30
OK
mini-redis> SET user:1 "Alice"
OK
mini-redis> SET user:2 "Bob"
OK
mini-redis> SET user:3 "Charlie"
OK

# maxmemory(30) 초과로 LRU(user:1) 제거

mini-redis> GET user:1
(nil)
mini-redis> INFO memory
used_memory:22
maxmemory:30
evicted_keys:1
mini-redis> KEYS
1. "user:2"
2. "user:3"
mini-redis> EXPIRE user:2 3
(integer) 1
mini-redis> TTL user:2
(integer) 2

# (3초 경과 후)

mini-redis> GET user:2
(nil)
mini-redis> TTL user:2
(integer) -2
```

- 에러 출력 예시(예)

```CSS
mini-redis> CONFIG SET maxmemory abc
(error) ERR value is not an integer or out of range
mini-redis> GET
(error) ERR wrong number of arguments for 'GET' command
```

```
mini-redis> HELLO
(error) ERR unknown command 'HELLO'
```